

C++ Problem Statements

for

Siemens Gamesa / Siemens Energy

Wind Turbine & Power Generation Domain

C++ and Advanced C++ (without Design Patterns)

Classes, Objects, Encapsulation, Polymorphism, ABC, Inheritance

Templates, STL, Google Test Framework

Project Structure: bin/ inc/ obj/ src/ libs/ Makefile

Confidential - For Internal Training Use Only

Problem Statement 1: Wind Turbine SCADA Data Acquisition & Monitoring System

Background

Siemens Gamesa operates thousands of onshore and offshore wind turbines globally. Each turbine generates real-time operational data -- rotor speed, generator RPM, nacelle temperature, wind speed, pitch angle, gearbox oil temperature, and power output. The central SCADA (Supervisory Control and Data Acquisition) system collects and processes this data to detect anomalies, log events, and trigger alarms.

Currently, the plant operators rely on spreadsheet-based logging. This is error-prone and does not scale. You are tasked with building a console-based SCADA data processing engine in C++ that can ingest sensor readings from multiple turbine types, apply threshold-based alarm logic, and generate summary reports.

Requirements

Functional Requirements

1. The system must support three types of wind turbines -- Onshore (SG 5.0-145), Offshore Direct Drive (SG 14-222 DD), and Offshore Geared (SG 8.0-167 DD). Each turbine type has different operational envelopes (e.g., cut-in wind speed, rated power, cut-out wind speed).
2. Each turbine must expose a common interface for: accepting a sensor reading (wind speed, rotor speed, nacelle temperature, gearbox oil temperature, power output); determining whether the reading is within operational bounds; returning the turbine's rated capacity and status (Normal, Warning, Critical).
3. The system must maintain a real-time data buffer (a fixed-size circular buffer) of the last N readings per turbine. When a new reading arrives and the buffer is full, the oldest reading is overwritten.
4. The system must compute statistical summaries over the buffered data: Minimum, Maximum, Mean, Standard Deviation for each sensor parameter. The summary must be computed using a template-based statistics utility that works for any numeric type (int, float, double).
5. Alarm Logic: Warning -- Nacelle temperature > 70 degrees C OR gearbox oil temperature > 90 degrees C OR power output > 105% of rated capacity. Critical -- Nacelle temperature > 85 degrees C OR gearbox oil temperature > 110 degrees C OR rotor speed > 120% of rated RPM. When a Critical alarm is triggered, the system must log the event with a timestamp and the turbine ID.
6. The system must support sorting turbines by their average power output (descending) for a dashboard-style display.
7. The system must support filtering turbines that are in Warning or Critical state, using a predicate-based approach.

Non-Functional Requirements

8. Use abstract base classes (ABC) to define the turbine interface so that adding a new turbine type in the future does not require modifying existing code.
9. Use encapsulation -- all internal state (sensor readings, alarm history) must be private, accessed only through public methods.
10. Use STL containers (std::vector, std::map, std::deque) for managing turbine collections and data buffers.
11. Use templates for the statistics computation utility.
12. Use polymorphism -- the main processing loop must work with pointers/references to the base turbine type, not concrete types.
13. Project must follow the folder structure: bin/, inc/, src/, obj/, libs/, and a Makefile.
14. Unit tests must be written using Google Test covering at least: statistics computation for various types,

alarm threshold logic, circular buffer overflow behavior, sorting and filtering of turbines.

Sample Data for Testing

Turbine ID: WT-001, Type: Onshore SG 5.0-145

Reading 1: Wind=12.3 m/s, Rotor=14.2 RPM, NacelleTemp=62C, GearboxOil=78C, Power=4.8 MW

Reading 2: Wind=15.1 m/s, Rotor=15.0 RPM, NacelleTemp=74C, GearboxOil=92C, Power=5.3 MW [Warning]

Reading 3: Wind=18.7 m/s, Rotor=16.8 RPM, NacelleTemp=87C, GearboxOil=112C, Power=5.6 MW [Critical]

Turbine ID: WT-002, Type: Offshore SG 14-222 DD

Reading 1: Wind=9.5 m/s, Rotor=8.1 RPM, NacelleTemp=45C, GearboxOil=N/A, Power=10.2 MW

Reading 2: Wind=11.2 m/s, Rotor=9.3 RPM, NacelleTemp=48C, GearboxOil=N/A, Power=13.8 MW

Problem Statement 2: Wind Farm Power Forecasting & Revenue Estimation Module

Background

Siemens Gamesa's energy division must provide power generation forecasts to grid operators and energy traders. Accurate forecasting directly impacts revenue -- under-forecasting means leaving revenue on the table; over-forecasting leads to imbalance penalties from the grid operator. Each wind farm consists of multiple turbines with different capacities and availability factors.

You are tasked with building a power forecasting and revenue estimation engine that models a wind farm, predicts energy output over configurable time horizons, and estimates revenue based on market pricing tiers.

Requirements

Functional Requirements

1. Model a Wind Farm as a collection of turbines. Each turbine has: a unique ID, manufacturer model, rated capacity (MW); an availability factor (0.0 to 1.0) representing uptime probability; a power curve -- a mapping of wind speed bins (m/s) to expected power output (MW). This must be stored and queried efficiently.
2. Implement a power curve interpolation function (template or function object) that, given a wind speed, returns the expected power output by linearly interpolating between known data points in the power curve.

Wind Speed (m/s):	3	5	7	9	11	13	15	17	19	21	23	25
Power Output (MW):	0.0	0.3	1.2	2.5	3.8	4.6	4.9	5.0	5.0	4.8	2.0	0.0

3. Implement a Forecast Engine that: takes as input a list of wind speed forecasts (time-stamped predictions for the next N hours, hourly resolution); for each turbine, computes the expected power output at each hour using the power curve; applies the availability factor to get net expected output; aggregates across all turbines to produce a farm-level hourly forecast; computes total energy (MWh) over the forecast horizon.
4. Implement a Revenue Estimator that: takes the farm-level hourly forecast and a pricing schedule (different \$/MWh rates for peak, off-peak, and shoulder hours); classifies each hour into peak/off-peak/shoulder based on hour of day (peak: 08:00-20:00, off-peak: 22:00-06:00, shoulder: remaining); computes total estimated revenue; supports applying a curtailment penalty -- if the farm over-forecasts by more than 15% compared to actual (simulated) output, a penalty of \$50/MWh is applied to the excess.
5. Data persistence for one session: All forecasts and revenue estimates must be stored in an in-memory structure and retrievable. Implement an operator<< overload to serialize the full forecast report to an output stream.
6. Support comparing two forecast scenarios -- e.g., 'forecast with all turbines at 95% availability' vs. 'forecast with 2 turbines offline' -- and output the delta in total energy and revenue.

Non-Functional Requirements

7. Use ABC for a ForecastStrategy abstraction -- the system should allow different forecasting strategies (e.g., naive forecast vs. weighted average forecast) to be plugged in without modifying the engine.
8. Use templates for the power curve interpolation to support different numeric precisions.
9. Use STL -- std::map for power curves, std::vector for time-series data, std::algorithm for transformations.

10. Use inheritance and polymorphism -- the revenue estimator should work with any forecast result type through a base interface.
11. Encapsulation -- wind farm internals (individual turbine data) must not be directly accessible from outside; provide aggregated views only.
12. Follow the folder structure: bin/, inc/, src/, obj/, libs/, Makefile.
13. Google Test coverage for: power curve interpolation (boundary values, mid-range, extrapolation); forecast computation for a single turbine; farm-level aggregation; revenue calculation with pricing tiers; curtailment penalty logic; scenario comparison delta.

Sample Data

Wind Farm: "Baltic Eagle"

Turbine 1: SG 14-222 DD, Rated: 14 MW, Availability: 0.97

Turbine 2: SG 14-222 DD, Rated: 14 MW, Availability: 0.92

Turbine 3: SG 14-222 DD, Rated: 14 MW, Availability: 1.00

Hourly Wind Forecast (next 6 hours):

H1: 11.2 m/s, H2: 13.5 m/s, H3: 15.8 m/s

H4: 14.1 m/s, H5: 10.3 m/s, H6: 8.7 m/s

Pricing: Peak=\$85/MWh, Shoulder=\$62/MWh, Off-Peak=\$41/MWh

Problem Statement 3: Wind Turbine Component Lifecycle & Maintenance Scheduler

Background

Wind turbine maintenance is one of the largest operational costs for Siemens Gamesa. Each turbine has multiple critical subsystems -- rotor blades, gearbox, generator, pitch system, yaw system, and tower structure. Each subsystem has a different expected lifespan, degradation rate, and failure consequence. Unscheduled maintenance (breakdowns) costs 3-5x more than planned maintenance.

You are tasked with building a component lifecycle tracking and maintenance scheduling system that tracks component health, predicts remaining useful life (RUL), and generates an optimized maintenance schedule for a fleet of turbines.

Requirements

Functional Requirements

1. Model subsystems as a class hierarchy: Base class Component -- has name, installation date, rated lifetime (hours of operation), current operating hours, maintenance history. Derived classes: Gearbox, Generator, BladeSet, PitchSystem, YawSystem, TowerStructure.
2. Each derived class must implement a health calculation method (polymorphic) that returns a health factor (0.0 = failed, 1.0 = new). The health calculation logic differs per component:

```
Gearbox:      health = 1.0 - (hours/rated) * (1 + 0.001 * temp_cycles)
Generator:    health = 1.0 - (hours/rated)^1.5
BladeSet:     health = 1.0 - (hours/rated) * erosion_factor
PitchSystem:  health = 1.0 - (hours/rated) * (1 + 0.005 * num_faults)
YawSystem:    health = 1.0 - (hours/rated) * 0.8
TowerStructure: health = 1.0 - (hours/rated) * 0.3
```

3. Maintenance Events must be tracked as records containing: component ID, date, type (Scheduled / Unscheduled / Inspection), cost, duration (hours), technician count. A history of maintenance events must be stored per component using an STL container.
4. Maintenance Scheduling Engine: For each component, compute Remaining Useful Life (RUL) = rated_lifetime - current_operating_hours (adjusted by health factor). Generate a maintenance schedule listing all components needing maintenance within the next N operating hours (user-configurable). Sort by urgency (lowest RUL first). Flag health < 0.2 as 'Replace Immediately' and 0.2-0.4 as 'Schedule within 500 hours'.
5. Cost Estimator: Given a maintenance schedule, estimate total maintenance cost. Scheduled maintenance uses a base cost per component type (configurable). Unscheduled (breakdown) cost is 3.5x the scheduled cost. The system must compare proactive vs. reactive maintenance costs and output both scenarios.
6. Fleet Dashboard: Generate a report showing per-turbine health score (average of all subsystem health values); per-turbine maintenance cost-to-date; fleet-wide statistics: turbines in 'Good' (avg health > 0.7), 'Caution' (0.4-0.7), 'Critical' (< 0.4). Use STL algorithms and containers for aggregation.

Non-Functional Requirements

7. Use ABC for the Component base class -- make the health calculation a pure virtual function.
8. Use templates for a generic cost estimation function that works with different currency types or cost models (e.g., CostEstimator<CostModel>).
9. Use STL extensively -- std::vector, std::sort, std::find_if, std::accumulate, std::map, std::priority_queue for

the scheduling queue.

10. Use encapsulation -- component health details must be accessed through controlled interfaces; raw data members must be private.
11. Use operator overloading -- operator< for components (by RUL) to enable direct sorting; operator+ to aggregate maintenance costs; operator<< for printing component status.
12. Follow folder structure: bin/, inc/, src/, obj/, libs/, Makefile.
13. Google Test coverage for: health calculation for each component type (known inputs, expected outputs); RUL computation; maintenance schedule generation and sorting; cost estimation (scheduled vs. unscheduled scenarios); fleet dashboard categorization; edge cases: brand-new component (0 hours), component past rated lifetime.

Sample Data

Turbine: WT-OFF-007 (Offshore, Baltic Sea)

Gearbox:	Installed 2021-03-15, Rated: 100,000 hrs, Current: 32,500 hrs
Generator:	Installed 2021-03-15, Rated: 150,000 hrs, Current: 32,500 hrs
BladeSet:	Installed 2021-03-15, Rated: 200,000 hrs, Current: 32,500 hrs
PitchSystem:	Installed 2021-06-01, Rated: 80,000 hrs, Current: 31,000 hrs, Faults: 14
TowerStruct:	Installed 2021-03-15, Rated: 300,000 hrs, Current: 32,500 hrs

Scheduled maintenance costs:

Gearbox:	\$45,000	Generator:	\$28,000	BladeSet:	\$15,000
PitchSystem:	\$8,000	YawSystem:	\$5,000	Tower:	\$12,000

Problem Statement 4: Alarm Correlation & Root Cause Analysis Engine

Background

A single wind turbine can generate 500+ alarm signals per day. Many of these alarms are cascading -- a single root cause (e.g., gearbox bearing failure) triggers secondary alarms (high temperature, vibration, oil pressure loss) across multiple subsystems. Operators are overwhelmed by alarm floods and cannot quickly identify the root cause. This leads to delayed response, increased downtime, and higher maintenance costs.

Siemens Gamesa needs an alarm correlation and root cause analysis (RCA) engine that ingests raw alarm streams, deduplicates repeated alarms, correlates related alarms into alarm groups, and suggests probable root causes based on historical patterns.

Requirements

Functional Requirements

1. Model Alarms with the following attributes: Alarm ID (unique), source turbine ID, subsystem (Gearbox / Generator / Pitch / Yaw / Blade / Tower / BOP), severity (Info / Warning / Failure / Critical), timestamp, message, acknowledged (bool).
2. Model Alarm Correlation Rules that define which alarms can cascade from a root cause. Example: Gearbox Bearing Failure (Critical) can cascade to Gearbox High Temperature (Warning), Gearbox Oil Pressure Low (Failure), Gearbox Vibration High (Warning), Generator Speed Fluctuation (Warning). Rules must be configurable and stored in an STL container (e.g., `std::map<AlarmType, std::vector<AlarmType>>`).
3. Alarm Deduplication Engine: If the same alarm (same source, same subsystem, same type) fires multiple times within a configurable debounce window (e.g., 60 seconds), only the first occurrence is kept, and the rest are marked as duplicates with a count. The debounce window should be a template parameter so it can be instantiated for different time granularities (seconds, minutes).
4. Alarm Correlation Engine: Given a time window (e.g., all alarms within a 5-minute window for a single turbine), group related alarms into Alarm Groups using the correlation rules. An Alarm Group has: a suggested root cause, a list of correlated alarms, a combined severity (highest severity among the group), and a confidence score (based on how many expected cascading alarms are present vs. total defined in the rule).
5. Root Cause Suggestion: For each alarm group, rank possible root causes by confidence score. Confidence > 80% = High Confidence, 50-80% = Medium Confidence, < 50% = Low Confidence. Output the top-3 suggested root causes per alarm group.
6. Historical Pattern Learning (simple, rule-based): Maintain a log of all alarm groups and their resolved root causes (operator confirms or corrects the suggested root cause). For future alarms, if the same pattern (same subsystem, similar alarm sequence) has been seen before with a confirmed root cause, boost the confidence of that root cause by 10%. Store the history in an STL container and use `std::find_if` / `std::count_if` to search patterns.
7. Report Generation: For a given turbine and time range, produce: total alarms received, total after deduplication, total alarm groups formed; top 5 most frequent alarm patterns; average time between first alarm in a group and operator acknowledgment. All report data must be printable via `operator<<` overloads.

Non-Functional Requirements

8. Use ABC for alarm sources -- different turbine types may emit different alarm schemas, but the correlation engine must work with any alarm source through a common interface.
9. Use templates for the deduplication debounce mechanism (parameterized by time unit).
10. Use STL -- std::map, std::vector, std::set, std::priority_queue (for severity ranking), std::algorithm for sorting/filtering alarm groups.
11. Use inheritance for alarm severity levels or subsystem types if modeling makes sense.
12. Use encapsulation -- alarm internal state (raw timestamps, debounce counters) must be private; provide read access through const member functions.
13. Follow folder structure: bin/, inc/, src/, obj/, libs/, Makefile.
14. Google Test coverage for: alarm deduplication within and outside debounce window; alarm group formation with known correlation rules; confidence score calculation (full match, partial match, no match); root cause ranking; historical pattern boosting; edge cases: single alarm (no correlations), alarm flood (100+ alarms in window), alarms spanning multiple subsystems with no rules defined.

Sample Data

Turbine: WT-OFF-012, Time Window: 10:00:00 to 10:05:00

```
10:00:01 [Failure] Gearbox: Gearbox oil pressure low
10:00:03 [Warning] Gearbox: Gearbox temperature high
10:00:05 [Warning] Gearbox: Gearbox vibration high
10:00:08 [Warning] Generator: Generator speed fluctuation detected
10:00:09 [Critical] Gearbox: Gearbox bearing failure imminent <-- root cause
10:00:12 [Info] BOP: Grid frequency deviation (minor)
10:00:15 [Warning] Gearbox: Gearbox temperature high <-- duplicate
10:00:18 [Info] Pitch: Pitch system self-test initiated
10:04:55 [Failure] Gearbox: Gearbox oil pressure low <-- new occurrence
```

Problem Statement : Summary Table

#	Domain Focus	Key C++ Concepts Emphasized
1	SCADA Data Acquisition	ABC, polymorphism, templates (stats), STL (circular buffer, map), encapsulation
2	Power Forecasting & Revenue	Templates (interpolation), ABC (forecast strategy), operator overloading, STL
3	Component Lifecycle & Maint.	Deep inheritance, ABC, templates (cost model), operator overloading, STL
4	Alarm Correlation & RCA	ABC, templates (dedup debounce), inheritance, STL (map, set, priority_queue)